

Protocolo de Trabajo con Git, GitHub y Linear

Taller de Integración II y Taller de Integración IV

Segundo semestre 2026

1. Objetivo

El presente documento define la metodología de trabajo que utilizarán los equipos de Taller de Integración II (TI2) y Taller de Integración IV (TI4) durante el desarrollo conjunto del proyecto.

El objetivo es establecer un flujo de trabajo simple, trazable y consistente, que permita que ambos equipos colaboren sobre un mismo producto, manteniendo responsabilidades técnicas diferenciadas.

Las herramientas principales serán:

- **Linear**: planificación, asignación y seguimiento del trabajo.
- **Git**: control de versiones.
- **GitHub**: alojamiento del código, Pull Requests, revisión de código e integración continua.

El principio general será:

Linear gestiona el trabajo y GitHub gestiona el código.

2. Organización de los equipos

El proyecto estará compuesto por dos equipos.

2.1. Taller de Integración II

El equipo TI2 estará compuesto por seis estudiantes y será responsable principalmente de:

- desarrollo de la aplicación web;
- desarrollo del backend y API;
- diseño e implementación de la base de datos;
- pruebas asociadas a sus componentes;
- despliegue de sus propios servicios;
- documentación técnica correspondiente.

2.2. Taller de Integración IV

El equipo TI4 estará compuesto por tres estudiantes y será responsable principalmente de:

- desarrollo de la aplicación móvil;
- planificación general del proyecto;
- preparación y priorización inicial del trabajo;
- coordinación entre ambos equipos;
- control de calidad;

- revisión de código;
- seguimiento de dependencias;
- apoyo técnico e inducción al equipo TI2 cuando sea necesario.

TI4 deberá procurar que TI2 pueda resolver progresivamente sus propias tareas. El objetivo de las revisiones no será corregir directamente el código de otro integrante, sino entregar retroalimentación que permita que su autor realice las modificaciones correspondientes.

3. Repositorio GitHub

Se utilizará **un único repositorio GitHub** para todo el producto. El repositorio tendrá una estructura de monorepo que permitirá mantener las aplicaciones, el backend y los paquetes compartidos bajo el mismo control de versiones.

```
Proyecto
|
+-- apps/
| +-- web/ Aplicación web de TI2
| +-- mobile/ Aplicación móvil de TI4
|
+-- convex/ Backend y base de datos compartidos
+-- packages/ Código compartido cuando corresponda
+-- .github/workflows/ Integración continua
+-- package.json
+-- bun.lock
```

Cada equipo será responsable del desarrollo, pruebas y despliegue de los componentes que tenga asignados dentro del monorepo.

La existencia de un solo repositorio no elimina la separación de responsabilidades: TI2 mantiene la aplicación web y el backend compartido, mientras TI4 mantiene la aplicación móvil.

La principal frontera de integración entre ambos equipos será la API pública de Convex, cuyos tipos generados podrán ser consumidos directamente por ambas aplicaciones sin copiar implementaciones ni mantener contratos externos.

4. Uso de Linear

Se utilizará **un único workspace de Linear** para todo el proyecto.

Dentro de este workspace existirán dos Teams:

- **TI2:** Web, Backend y Base de Datos.
- **TI4:** Gestión y Aplicación Móvil.

Ambos Teams representan partes del mismo proyecto.

4.1. Triage

TI4 será responsable de realizar inicialmente el *triage* del trabajo.

Esto incluye:

- crear o revisar los issues necesarios;
- asignarlos al Team correspondiente;
- definir su prioridad;
- verificar que tengan una descripción suficiente;
- establecer criterios de aceptación;

- identificar dependencias entre ambos equipos;
- asignarlos al Cycle correspondiente.

Una vez que el trabajo se encuentre preparado en el backlog, cada equipo podrá organizar internamente su ejecución.

4.2. Linear como fuente de verdad

Toda tarea técnica que requiera trabajo significativo debe existir en Linear.

No se considerarán como mecanismo formal de asignación de trabajo mensajes aislados enviados por WhatsApp, Discord u otros canales.

Estos medios podrán utilizarse para comunicación, pero la tarea deberá quedar registrada en Linear.

4.3. Sub-issues

No se utilizarán **sub-issues** de Linear.

Cuando una tarea necesite varios pasos pequeños, estos se registrarán mediante una lista dentro de la descripción del issue.

Por ejemplo:

TI2-37 Implementar autenticación

- Crear endpoint de login
- Validar credenciales
- Generar token
- Agregar pruebas
- Documentar endpoint

Si una parte del trabajo tiene suficiente independencia como para requerir asignación, seguimiento o revisión propia, entonces deberá convertirse en un issue independiente.

Esta regla también permite administrar de mejor manera el límite de issues disponible en el plan utilizado durante el curso.

5. Cycles y Sprints

Los *Cycles* de Linear representarán los Sprints del proyecto.

Por lo tanto, no se utilizará un Cycle diferente para cada semana.

Conceptualmente:

Cycle 1 -> Sprint 1
 Cycle 2 -> Sprint 2
 Cycle 3 -> Sprint 3
 Cycle 4 -> Sprint Final

Las semanas existentes dentro de un Sprint se utilizarán para seguimiento, reuniones y control de avance.

Al finalizar un Cycle se deberá revisar:

- trabajo completado;
- trabajo pendiente;
- issues bloqueados;
- deuda técnica generada;
- dependencias entre TI2 y TI4;
- trabajo que debe trasladarse al siguiente Sprint.

6. Estados de los Issues

Se utilizará un workflow reducido.

Backlog → Todo → In Progress → In Review → Done

También podrá existir el estado:

Canceled

Cada estado representa:

Backlog: trabajo identificado pero todavía no comprometido.

Todo: trabajo seleccionado para el Cycle actual.

In Progress: existe trabajo activo sobre el issue.

In Review: existe uno o más Pull Requests pendientes de revisión.

Done: todo el trabajo requerido por el issue fue integrado.

Canceled: el trabajo dejó de ser necesario.

Siempre que sea posible, los cambios de estado asociados al desarrollo deberán automatizarse mediante la integración Linear–GitHub.

7. Integración de Linear con GitHub

El repositorio GitHub del monorepo deberá conectarse al workspace de Linear.

Los issues conservarán el Team correspondiente, TI2 o TI4, y su identificador permitirá distinguir el área responsable aunque todos los cambios vivan en el mismo repositorio.

Los identificadores de Linear serán utilizados para relacionar automáticamente el código con el trabajo correspondiente.

Por ejemplo:

TI2-37 TI4-18

Un Pull Request puede hacer referencia al issue correspondiente mediante su branch, título o descripción.

De esta forma será posible recorrer la trazabilidad:

Issue Linear → Branch → Commits → Pull Request → Review → Merge

8. Estrategia de ramas

El repositorio utilizará una estrategia basada en **feature branches pequeñas y de corta duración**.

La única rama permanente será:

main

No se utilizará una rama permanente **develop**.

Tampoco se utilizará Git Flow completo con ramas permanentes de desarrollo, release o hotfix.

8.1. Creación de una rama

Antes de comenzar una tarea:

1. verificar que exista un issue en Linear;
2. actualizar la rama `main`;
3. crear una nueva rama desde `main`;
4. desarrollar solamente el objetivo asociado al issue.

Flujo conceptual:

```
main
|
+-- rama TI2-37-login-web
|
+-- rama TI2-42-auth
|
+-- rama TI4-18-login-mobile
```

9. Nombres de ramas

Se utilizará preferentemente el nombre de rama generado desde Linear.

El formato deberá conservar:

- nombre o identificador del desarrollador;
- identificador del issue de Linear;
- una descripción breve.

Ejemplo:

```
vrivera/ti2-37-login
```

Si Linear genera automáticamente un nombre excesivamente largo, este podrá abreviarse manualmente.

Por ejemplo, en lugar de:

```
vrivera/ti2-37-implementar-endpoint-de-autenticacion-de-usuarios
```

se podrá utilizar:

```
vrivera/ti2-37-login
```

El identificador de Linear debe conservarse.

No se deberá utilizar:

```
login
feature1
cambios
prueba
rama-nueva
```

10. Tamaño de las ramas

Una rama debe resolver **una intención claramente identificable**.

Las ramas deberán mantenerse pequeñas para facilitar:

- revisión;
- pruebas;

- resolución de conflictos;
- integración continua;
- detección de errores;
- reversión de cambios.

No se establecerá un límite artificial de líneas modificadas.

El criterio será que un Pull Request pueda ser entendido y revisado sin incluir múltiples funcionalidades independientes.

11. Pull Requests

Todo cambio que deba incorporarse a **main** deberá pasar por un **Pull Request** en el repositorio del monorepo.

No se permitirá realizar push directo a **main**.

El Pull Request deberá indicar:

- qué problema resuelve;
- qué cambios introduce;
- cómo puede probarse;
- issue de Linear relacionado;
- cualquier consideración relevante para el reviewer.

Ejemplo:

```
TI2-37: Implementar login de usuarios
```

La descripción deberá permitir que otra persona pueda comprender el cambio sin tener que reconstruir todo el contexto desde el código.

12. Relación entre Issues y Pull Requests

No existe una obligación de mantener una relación 1:1 entre un issue y un Pull Request.

Un issue representa una **unidad de trabajo**.

Un Pull Request representa una **unidad de integración de código**.

Por lo tanto, un mismo issue puede requerir varios Pull Requests.

Ejemplo:

```
TI2-42 Autenticación
```

```
|
+-- PR 1: modelo de usuario
|
+-- PR 2: endpoint login
|
+-- PR 3: refresh token
```

El issue solamente se considerará terminado cuando todo el trabajo necesario haya sido integrado.

13. Stacked Pull Requests

Cuando una tarea no pueda dividirse completamente en cambios independientes, se podrán utilizar Pull Requests encadenados o *stacked PRs*.

Ejemplo:

```
main
|
+-- PR A
    |
    +-- PR B
        |
        +-- PR C
```

Esta técnica será excepcional y no el flujo predeterminado.

Como regla:

se recomienda un máximo de 3 stacked PRs y se permitirá un máximo de 4.

Si una cadena requiere más de cuatro Pull Requests dependientes entre sí, deberá revisarse la planificación y la división de la tarea.

14. Revisión de código

Todo Pull Request deberá ser revisado por al menos otra persona antes de ser integrado.

Las revisiones podrán terminar en:

- aprobación;
- comentarios;
- solicitud de cambios.

La revisión deberá enfocarse principalmente en:

- cumplimiento del issue;
- corrección;
- claridad del código;
- mantenibilidad;
- errores potenciales;
- pruebas;
- seguridad;
- compatibilidad con otros componentes;
- contrato de API cuando corresponda.

15. Progresión de las revisiones de TI2

Debido a que TI2 se encuentra en una etapa inicial de formación, el nivel de supervisión evolucionará durante el semestre.

Etapla inicial

Los Pull Requests de TI2 deberán ser revisados por integrantes de TI4.

TI4 podrá solicitar modificaciones, explicar problemas y recomendar posibles soluciones.

Sin embargo, el autor original deberá realizar los cambios en su propia rama.

Etapla intermedia

TI2 comenzará a realizar revisiones entre pares.

TI4 mantendrá revisión directa principalmente en cambios relacionados con:

- API;

- arquitectura;
- base de datos;
- seguridad;
- integración con móvil;
- cambios con impacto significativo.

Etapa avanzada

TI2 deberá ser capaz de revisar e integrar de manera más autónoma sus cambios.

TI4 mantendrá principalmente responsabilidades de:

- coordinación;
- QA;
- arquitectura;
- integración;
- resolución de dependencias.

16. Política de Merge

La estrategia estándar será:

Squash and Merge

Esto permitirá mantener una historia de `main` simple y legible aunque durante el desarrollo de una branch existan múltiples commits intermedios. Después del merge:

- la rama utilizada para el desarrollo **no deberá eliminarse**;
- el issue relacionado deberá quedar actualizado;
- cualquier documentación asociada deberá encontrarse incluida.

Las ramas ya integradas se conservarán como parte del historial de trabajo del proyecto y como evidencia de las actividades realizadas por cada integrante. Por lo tanto, deberá desactivarse en GitHub cualquier configuración que elimine automáticamente las ramas después de realizar un merge.

17. Protección de main

La rama `main` deberá protegerse mediante las opciones disponibles en GitHub.

Como mínimo:

- no permitir push directo;
- exigir Pull Request;
- exigir al menos una aprobación;
- exigir que las validaciones automáticas requeridas finalicen correctamente;
- exigir resolución de conversaciones pendientes antes del merge.

Estas reglas se aplicarán tanto a TI2 como a TI4.

Los integrantes de TI4 tampoco deberán evitar el proceso de Pull Request.

18. Integración Continua

El repositorio deberá utilizar GitHub Actions u otro mecanismo equivalente para ejecutar automáticamente verificaciones sobre los Pull Requests.

Las verificaciones dependerán de las rutas modificadas, pero deberán considerar como mínimo:

- instalar las dependencias desde el `package.json` y el único `bun.lock` de la raíz;
- ejecutar el build de la aplicación o paquete afectado;
- formatter;
- linter;
- pruebas automatizadas disponibles.

Cuando un cambio afecte el backend compartido, las aplicaciones Web y Mobile o un paquete compartido, la CI deberá ejecutar las verificaciones de todos los componentes afectados.

Un Pull Request con CI fallando no deberá integrarse a `main` salvo una situación excepcional debidamente justificada.

19. Integración entre TI2 y TI4

El equipo TI2 será responsable de implementar y desplegar los servicios compartidos del proyecto utilizando **Convex**. Convex proporcionará la capa de servidor y persistencia de datos utilizada por las distintas aplicaciones del proyecto. La autenticación se implementará utilizando **Better Auth**.

Al trabajar en un monorepo, TI2 y TI4 podrán modificar en un mismo Pull Request las partes de una funcionalidad que necesiten coordinación. También podrán utilizar Pull Requests separados cuando exista una razón clara de revisión o despliegue. En ambos casos, la relación entre los cambios deberá quedar registrada en Linear.

Cuando un cambio realizado por TI2 afecte una funcionalidad utilizada por TI4, deberá:

- quedar asociado a un issue de Linear;
- identificar al equipo afectado cuando corresponda;
- comunicar cambios incompatibles antes de integrarlos;
- mantener actualizada la documentación necesaria para consumir el servicio;
- incluir las modificaciones necesarias de autenticación, tipos o estructura de datos cuando corresponda.

20. Cambios en el modelo de datos

El almacenamiento de datos del proyecto será administrado mediante **Convex**. Los cambios realizados sobre el esquema, funciones, relaciones y estructuras de datos deberán mantenerse versionados en la carpeta `convex/` del monorepo. No deberán existir modificaciones necesarias para ejecutar el sistema que dependan exclusivamente de configuraciones manuales no documentadas.

Cuando una modificación del modelo de datos afecte funcionalidades utilizadas por TI4, esta deberá ser tratada como una dependencia entre equipos y quedar registrada en Linear.

21. Dependencias entre equipos

Cuando una tarea de un equipo dependa del trabajo del otro, esta dependencia deberá quedar registrada en Linear.

Ejemplo:

TI4-25 Pantalla de recuperación de clave

Bloqueada por:

TI2-61 Endpoint de recuperación de clave

TI4 será responsable de monitorear estas dependencias dentro del proceso de coordinación del proyecto.

22. Definición de Done

Un issue no se considerará terminado solamente porque “el código funciona en el computador del desarrollador”.

Como regla general, una tarea podrá marcarse como **Done** cuando:

- cumple sus criterios de aceptación;
- el código fue integrado mediante Pull Request;
- las revisiones fueron resueltas;
- CI finalizó correctamente;
- las pruebas necesarias fueron realizadas;
- la documentación fue actualizada cuando corresponde;
- no deja dependencias conocidas sin registrar;
- el cambio se encuentra disponible en **main**.

23. Flujo de trabajo completo

El flujo estándar será:

1. TI4 realiza el triage inicial del trabajo.
2. El issue queda en el Team y Cycle correspondiente.
3. El integrante toma una tarea desde Linear.
4. Actualiza localmente **main**.
5. Crea desde Linear una branch asociada al issue dentro del monorepo.
6. Implementa el cambio.
7. Realiza commits en su branch.
8. Publica la branch en GitHub.
9. Abre un Pull Request.
10. Linear relaciona el Pull Request con el issue.
11. El código pasa por CI.
12. Otro integrante realiza la revisión.
13. Si existen observaciones, el autor modifica su propia branch.
14. Cuando CI y review estén aprobados, se realiza Squash and Merge.
15. La branch utilizada se conserva en el repositorio.
16. Linear actualiza el estado del trabajo.
17. Cuando corresponda, el cambio es desplegado.

De forma resumida:

```
Linear
|
v
Issue
|
v
Branch pequeña
|
v
Desarrollo
|
v
```



24. Stack tecnológico

Ambos equipos deberán conocer el stack tecnológico del proyecto, incluso cuando cada uno sea responsable solamente de determinados componentes. Las principales tecnologías definidas para el proyecto son:

Componente	Tecnología seleccionada
Frontend Web	React + Vite + TanStack Router
Servidor / Backend	Convex
Base de Datos	Convex
Autenticación	Better Auth
Aplicación Móvil	Expo + React Native
Monorepo y dependencias	Bun workspaces
Integración Continua	GitHub Actions
Gestión de Proyecto	Linear
Control de Versiones	Git + GitHub

TI2 será responsable de desplegar y mantener los servicios compartidos, incluyendo la infraestructura implementada mediante Convex y la integración de autenticación con Better Auth. TI4 será responsable de integrar y mantener la aplicación móvil dentro del mismo monorepo. Las decisiones específicas de implementación, bibliotecas auxiliares y configuraciones internas podrán evolucionar durante el proyecto, siempre que no modifiquen los contratos y dependencias entre ambos equipos sin la coordinación correspondiente.

25. Principios generales

Durante el proyecto se aplicarán los siguientes principios:

1. Todo trabajo relevante debe ser trazable.
2. Nadie realiza push directo a **main**.
3. Las ramas deben ser pequeñas y de corta duración.
4. El código debe ser revisado por otra persona.
5. TI4 debe ayudar a TI2 a adquirir autonomía, no reemplazar su trabajo.
6. Linear representa el estado del trabajo.
7. GitHub representa el estado del código.

8. El monorepo contiene todos los componentes principales del producto.
9. La API compartida y los paquetes internos deben mantenerse explícitos y versionados.
10. Los problemas deben hacerse visibles tempranamente.

26. Resumen

La metodología utilizada puede resumirse mediante tres relaciones principales:

Linear = planificación y seguimiento

GitHub = código, revisión, CI e integración

Monorepo = código compartido, trazable y coordinado

El objetivo final no es solamente completar el producto, sino establecer un proceso de desarrollo colaborativo, reproducible y progresivamente autónomo para ambos equipos.